user clicks the submit button.

In the controller, the `order` variable is set to reference a new `Order` model object. This is done because the view populates the form from the data in this object. As it stands, that's not particularly interesting. Because it's a new model object, all the fields will be empty. However, consider the general case. Maybe we want to edit an existing order. Or maybe the user has tried to enter an order but the data has failed validation. In these cases, we want any existing data in the model shown to the user when the form is displayed. Passing in the empty model object at this stage makes all these cases consistent. The view can always assume it has a model object available. Then, when the user clicks the submit button, we'd like the new data from the form to be extracted into a model object back in the controller.

Fortunately, Rails makes this relatively painless. It provides us with a bunch of *form helper* methods. These helpers interact with the controller and with the models to implement an integrated solution for form handling. Before we start on our final form, let's look at a small example:

```
<%= form_with(model: order) do |form| %>
  <p>
    <%= form.label :name, "Name:" %>
    <%= form.text_field :name, size: 40 %>
  </p>
<% end %>
```

This code does two powerful things for us. First, the `form_with` helper on the first line sets up an HTML form that knows about Rails routes and models. The argument, `model: order`, tells the helper which instance variable to use when naming fields and sending the form data back to the controller.

The second powerful feature of the code is how it creates the form fields themselves. You can see that `form_with` sets up a Ruby block environment (that ends on the last line of the listing with the `end` keyword). Within this block, you can put normal template stuff (such as the `<p>` tag). But you can also use the block's parameter (`form` in this case) to reference a form context. We use this context to add a text field with a label by calling `text_field` and `label`, respectively.